

# ООП в Python: жизненный цикл объекта и dunder-методы

Краткая, но развёрнутая лекция о создании объекта в Python, специальных методах, управлении атрибутами и практических правилах проектирования простых учебных классов.

## Структура материала

1. Как в Python создаётся объект
2. Базовые dunder-методы и их назначение
3. Управление атрибутами и инвариантами
4. Равенство, хеширование и представление объекта
5. Практические рекомендации для учебных классов

## Как в Python создаётся объект

Создание экземпляра в Python делится как минимум на два шага. Сначала вызывается `__new__`, который отвечает за само создание объекта и выделение экземпляра, а затем `__init__`, который настраивает его состояние.

В большинстве учебных классов достаточно переопределять только `__init__`. Метод `__new__` нужен реже: например, для неизменяемых типов, паттерна singleton или тонкого контроля над созданием экземпляров.

Понимание этой последовательности важно при наследовании. Если базовый класс уже делает часть инициализации, дочерний класс должен согласованно вызывать `super()`, чтобы не нарушить инварианты объекта.

## КЛЮЧЕВЫЕ ТЕЗИСЫ

- `__new__` создаёт экземпляр и возвращает объект нужного класса.
- `__init__` получает уже созданный экземпляр и наполняет его данными.
- Если `__new__` возвращает объект другого типа, `__init__` текущего класса может не вызваться.
- При наследовании базовая инициализация должна вызываться явно через `super()`.

Минимальный класс с безопасной инициализацией

```
class Student:
    def __init__(self, name: str, course: int) -> None:
        self.name = name
        self.course = course
```

## Базовые dunder-методы и их назначение

Dunder-методы позволяют встроить пользовательский класс в поведение языка. Благодаря им объект умеет красиво печататься, сравниваться, храниться в множествах или реагировать на операции доступа к атрибутам.

На практике полезно связывать каждый специальный метод с конкретным сценарием. `__str__` нужен для дружелюбного вывода, `__repr__` — для разработчика и отладки, `__hash__` — для использования в `set` и `dict`, `__del__` — только в редких случаях, когда действительно требуется реакция на уничтожение объекта.

Не каждый метод нужно переопределять заранее. Лучше добавлять их по мере появления конкретной потребности, чтобы класс оставался простым и предсказуемым.

### КЛЮЧЕВЫЕ ТЕЗИСЫ

- `__str__` возвращает читаемое строковое представление объекта.
- `__repr__` ориентирован на разработчика и помогает в отладке.
- `__hash__` должен быть согласован с логикой равенства объектов.
- `__del__` не стоит использовать как основной механизм освобождения ресурсов.
- `__setattr__` даёт контроль над записью атрибутов, но требует аккуратности.

## Управление атрибутами и инвариантами

Метод `__setattr__` перехватывает каждую попытку записать атрибут. Это полезно для валидации, преобразования значений и защиты инвариантов, но любой неосторожный вызов `self.attr = ...` внутри него приведёт к бесконечной рекурсии.

Если нужно контролировать присваивание, внутри `__setattr__` следует использовать `object.__setattr__`. Тогда запись происходит напрямую, без повторного входа в пользовательский метод.

Для большинства учебных задач вместо тотального перехвата атрибутов удобнее использовать свойства через `@property`. Они точно валидируют поле и делают код легче для чтения.

### КЛЮЧЕВЫЕ ТЕЗИСЫ

- Инвариант — это правило, которое должно сохраняться для объекта после каждого публичного вызова.
- Нельзя напрямую писать `self.attr` внутри `__setattr__` без обхода через `object.__setattr__`.
- Свойства подходят для локальной валидации отдельных полей.
- Переопределение доступа оправдано, когда правило действительно относится ко всем изменениям состояния.

## Равенство, хеширование и представление объекта

Если объект должен сравниваться по содержимому, обычно переопределяют `__eq__`. После этого важно проверить, можно ли безопасно сделать объект хешируемым: изменяемые объекты редко подходят для использования как ключи словаря.

Строковое представление особенно помогает на этапе отладки и логирования. Хороший `__str__` показывает главные данные экземпляра без шума, а `__repr__` может быть более техническим и однозначным.

Для учебных моделей полезно отдельно проговорить, где важна идентичность объекта, а где — логическое равенство. Это снимает много типичных ошибок при работе с коллекциями.

### КЛЮЧЕВЫЕ ТЕЗИСЫ

- Если `__eq__` меняет критерий равенства, хеш должен следовать тем же правилам.
- Изменяемые объекты обычно не делают хешируемыми без особой причины.
- `__str__` отвечает на вопрос 'как показать объект пользователю'.
- `__repr__` отвечает на вопрос 'как показать объект разработчику'.

## Практические рекомендации для учебных классов

Хороший учебный класс не обязан поддерживать весь набор магических методов сразу. Гораздо полезнее сначала выделить данные, которые класс хранит, и операции, которые над ним действительно выполняются.

Если объект моделирует сущность предметной области, стоит отдельно продумать допустимые состояния и ошибки валидации. Это помогает писать компактный, но устойчивый код даже в простых лабораторных заданиях.

При подготовке к зачёту удобно держать в голове связку: создание объекта, человекочитаемое представление, валидация атрибутов и сравнение экземпляров. Эти четыре точки закрывают большую часть базовых вопросов по теме.

### КЛЮЧЕВЫЕ ТЕЗИСЫ

- Начинайте с простого `__init__` и добавляйте dunder-методы только по реальной необходимости.
- Используйте свойства для локальной валидации и ограниченного контроля доступа.
- Отдельно проверяйте совместимость `__eq__` и `__hash__`.
- Пишите классы так, чтобы их состояние было понятно по одному короткому выводу в логах.

Материал подготовлен в обезличенном виде на основе локальных учебных заметок и приведён к единому формату для SMHUB.